

Lab 4

Mohammadreza (Morez) Tayaranian

Modified by Matthew Mora

ECSE 444, Fall 2025

October 23

Topics

- I²C
- UART
- OS
- QSPI flash

When you enable OS and nothing works anymore



I²C

- What sensors do we have?

1. HTS221

- Temperature and Humidity

2. LIS3MDL

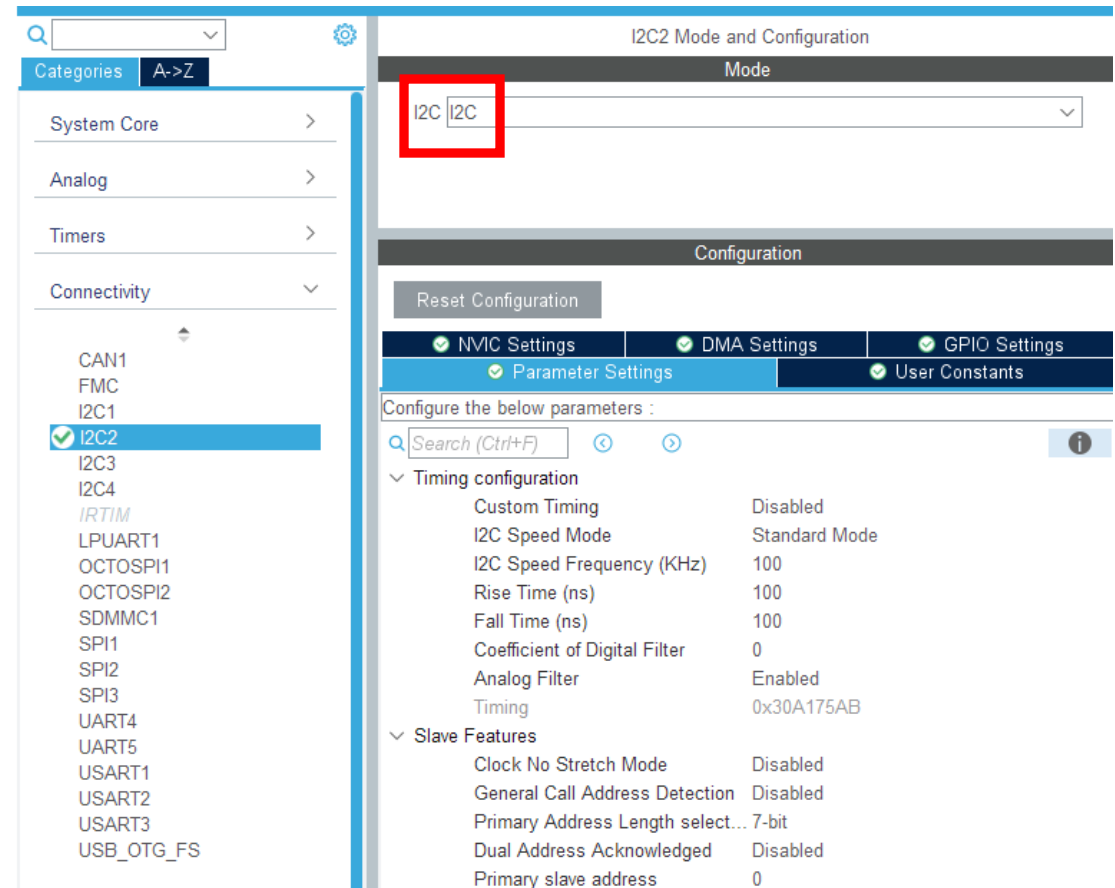
- Magnetometer

3. LSM6DSL

- Accelerometer and Gyroscope

4. LPS22HB

- Pressure



I²C

- We have BSP drivers that will read the data and convert it for us

1. Copy .h and .c files from

C:\Users\John\STM32Cube\Repository\STM32Cube_FW_L4_V
1.18.1\Drivers\BSP\B-L4S5I-IOT01

to the project's Core\Inc and Core\Src folders

I²C

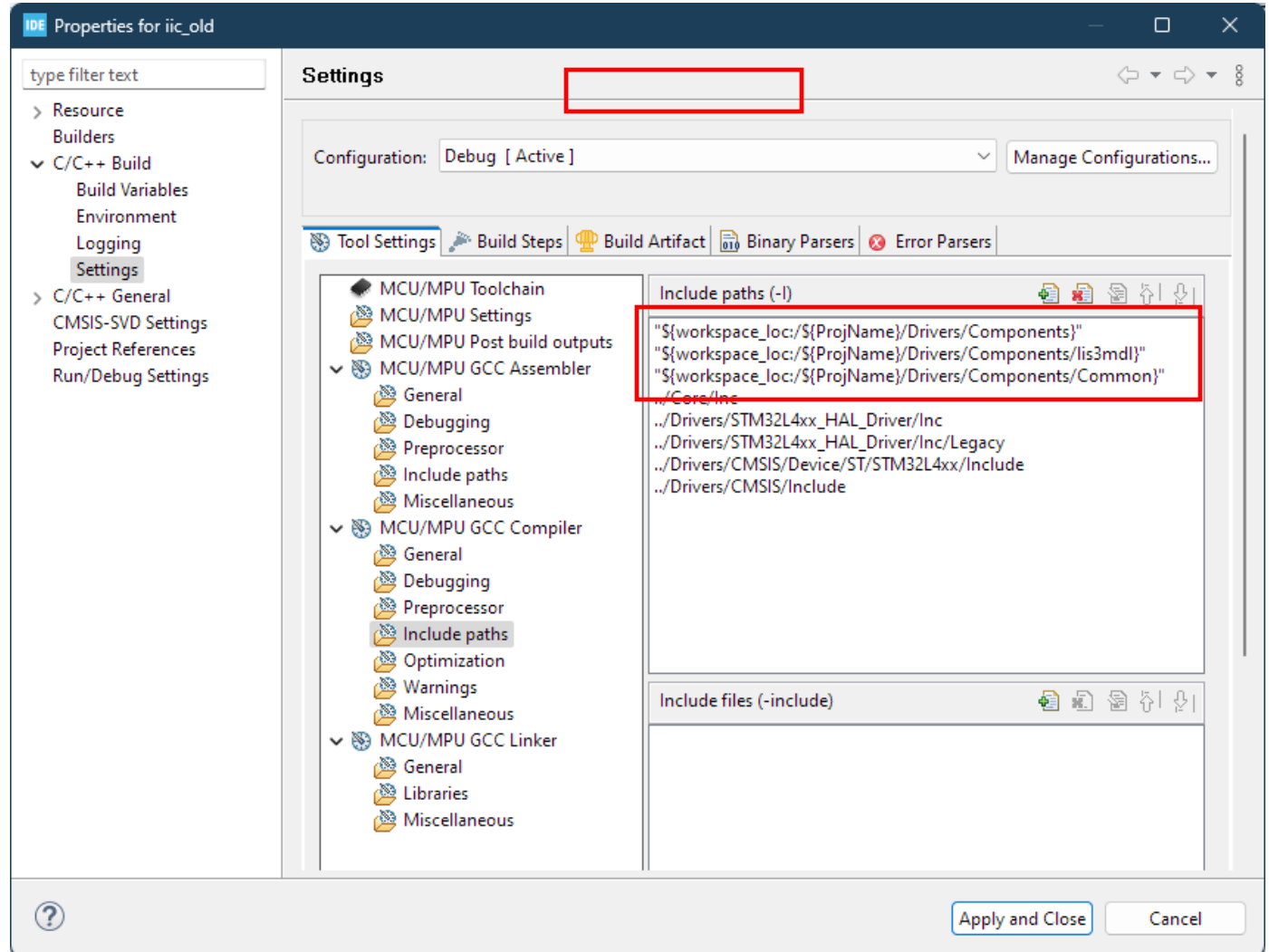
2. Copy the folder

C:\Users\John\STM32Cube\Repository\STM32Cube_FW_L4_V
1.18.1\Drivers\BSP\Components

to the project's Drivers folders

I²C

3. This step must be repeated for every new sensor you want to use. In this example we use LIS3MDL



I²C

4. Look at the header function you added in step 1 and use the functions to talk with the I2C sensor.
- This is an example for the humidity sensor
 - BSP_HSENSOR_Init
 - BSP_HSENSOR_ReadXXX

I²C

- While you probably won't need to know the addresses of each sensor, it can be helpful if you encounter issues.
- Can be found in this document: [Discovery kit for IoT node, multi-channel communication with STM32L4+ Series - User manual](#)
- An example from the document (you won't be using this sensor in the lab)

Modules	Description	SAD[6:0] + R/W	I2C write address	I2C read address
VL53L0X	Time-of-Flight ranging and gesture detection sensor	0101001x	0x52	0x53

UART

1. Enable it in ioc

*** Don't forget to make a note of the baud rate, this is important ***

Basic Parameters

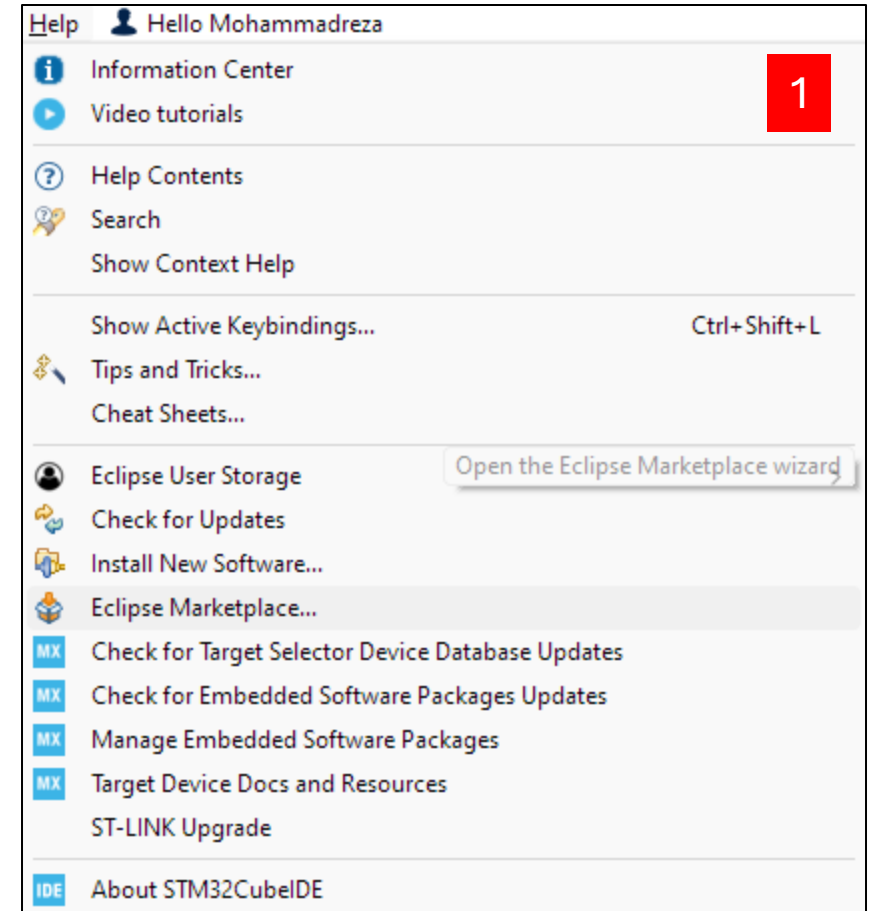
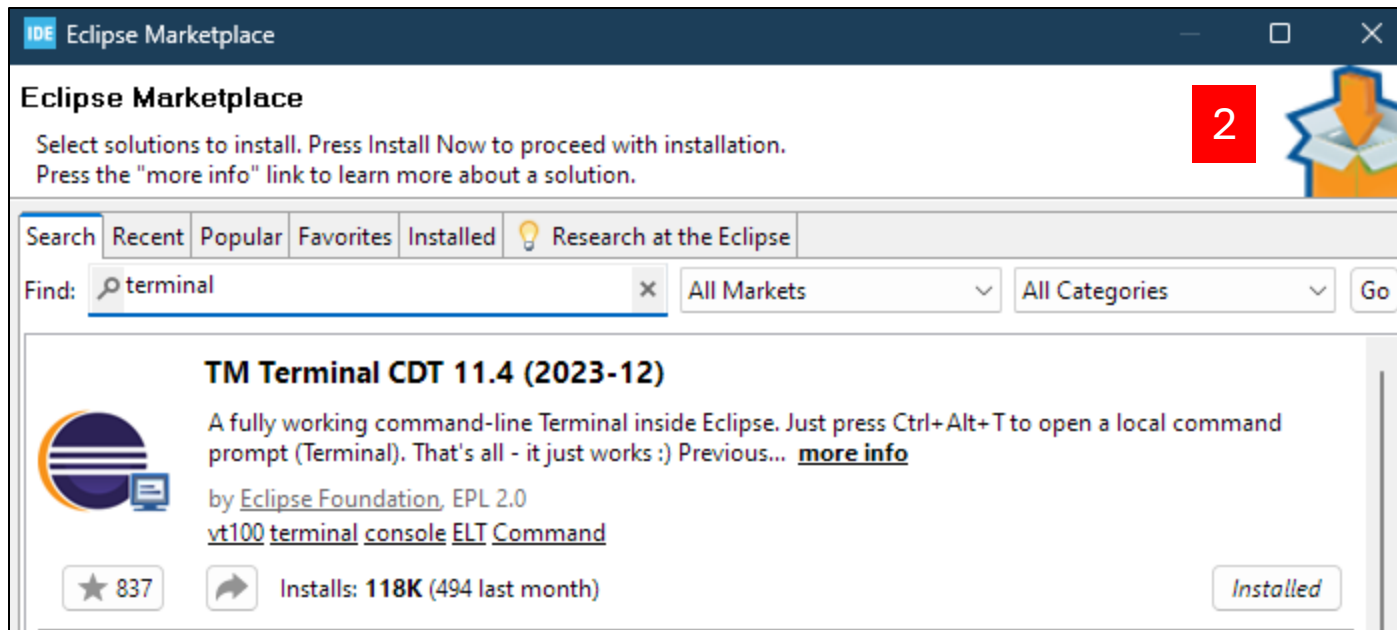
Baud Rate	115200 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1

The screenshot shows the STM32CubeMX Pinout & Configuration window. The left sidebar lists various components, with **USART1** selected and highlighted in blue. The main panel displays the configuration for USART1. The **Mode** is set to **Asynchronous**. The **Hardware Flow Control (RS232)** is set to **Disable**. The **Configuration** section includes a **Reset Configuration** button and tabs for **NVIC Settings**, **DMA Settings**, and **GPIO Settings**. The **Parameter Settings** tab is active, showing a search bar and a list of parameters. The **Basic Parameters** section includes **Baud Rate** (115200 Bits/s), **Word Length** (8 Bits (including Parity)), **Parity** (None), and **Stop Bits** (1). The **Advanced Parameters** section includes **Data Direction** (Receive and Transmit), **Over Sampling** (16 Samples), and **Single Sample** (Disable). The **Advanced Features** section includes **Auto Baudrate** (Disable), **TX Pin Active Level Inver...** (Disable), and **RX Pin Active Level Inver...** (Disable).

Pinout & Configuration		Clock Configuration	
Categories		Software Packs	
System Core		USART1 Mode and Configuration	
Analog		Mode	
Timers		Mode Asynchronous	
Connectivity		Hardware Flow Control (RS232) Disable	
CAN1		☐ Hardware Flow Control (RS485)	
FMC		Configuration	
I2C1		Reset Configuration	
I2C2		NVIC Settings	
I2C3		DMA Settings	
I2C3		GPIO Settings	
I2C3		Parameter Settings	
I2C3		User Constants	
I2C3		Configure the below parameters :	
I2C3		Search (Ctrl+F)	
I2C3		Basic Parameters	
I2C3		Baud Rate 115200 Bits/s	
I2C3		Word Length 8 Bits (including Parity)	
I2C3		Parity None	
I2C3		Stop Bits 1	
I2C3		Advanced Parameters	
I2C3		Data Direction Receive and Transmit	
I2C3		Over Sampling 16 Samples	
I2C3		Single Sample Disable	
I2C3		Advanced Features	
I2C3		Auto Baudrate Disable	
I2C3		TX Pin Active Level Inver... Disable	
I2C3		RX Pin Active Level Inver... Disable	

UART

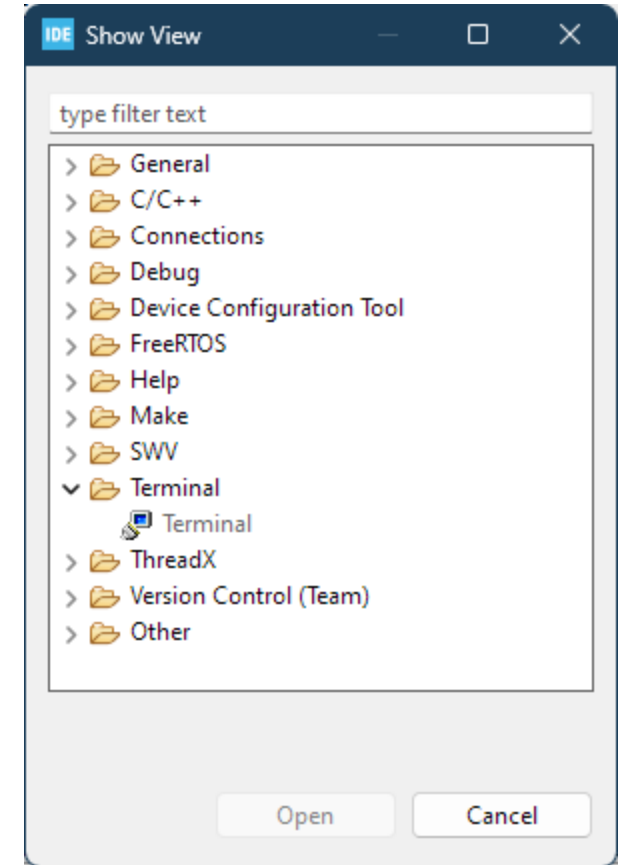
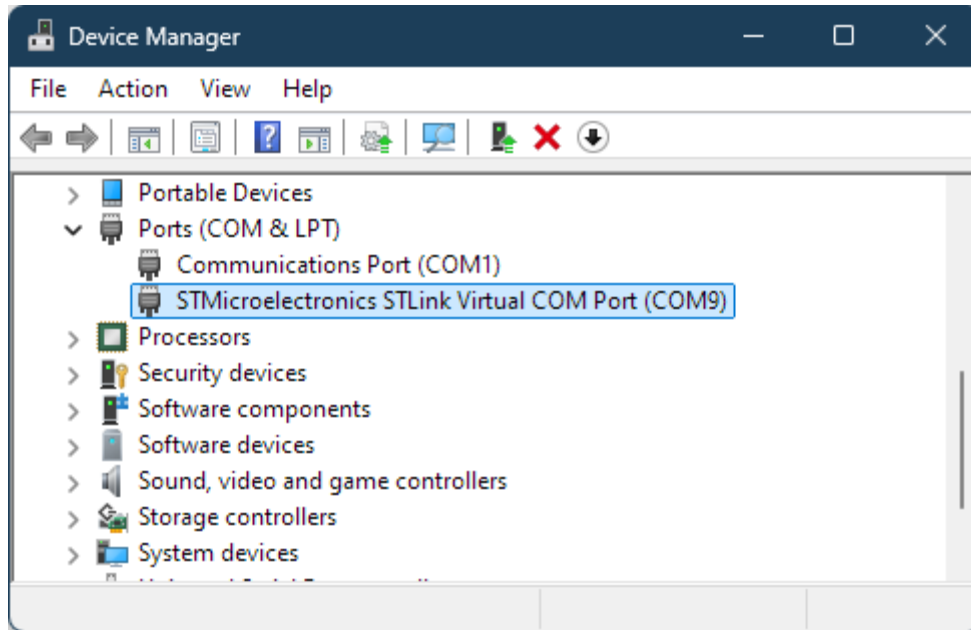
2. Install a terminal



UART

3. Open the terminal

- Make sure to connect to the correct COM port
- Also make sure the baud rates match



UART

4. Send data over UART

HAL_UART_Transmit

Function name	HAL_StatusTypeDef HAL_UART_Transmit (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)
Function description	Send an amount of data in blocking mode.
Parameters	• huart: UART handle. • pData: Pointer to data buffer. • Size: Amount of data to be sent. • Timeout: Timeout duration.
Return values	• HAL: status
Notes	• When FIFO mode is enabled, writing a data in the TDR register adds one data to the TXFIFO. Write operations to the TDR register are performed when TXFNF flag is set. From hardware perspective, TXFNF flag and TXE are mapped on the same bit-field.

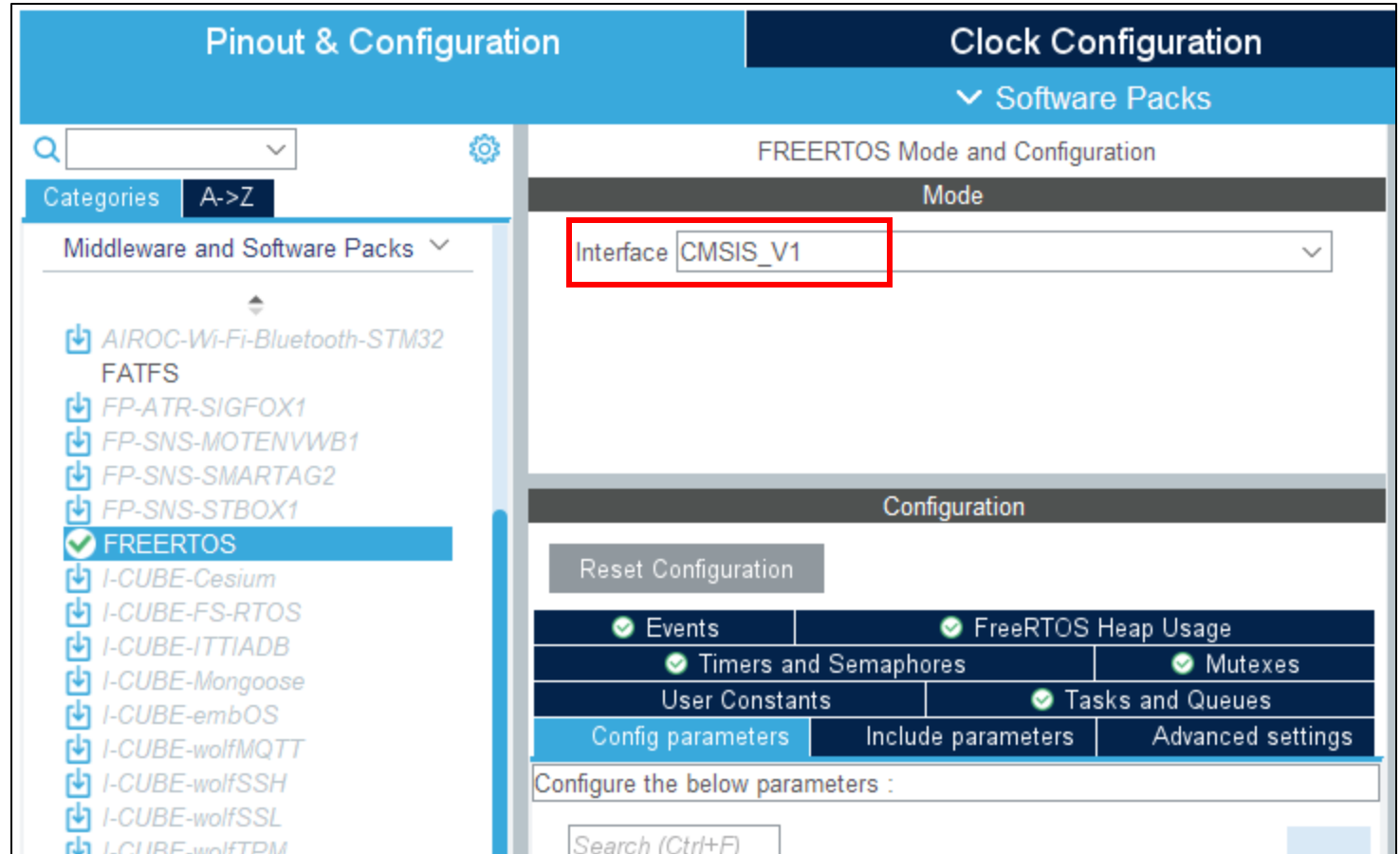
UART

- Example code

```
sprintf(output, "magDataXYZ = %d\r\n", data);  
uint16_t len = strlen(output);  
HAL_UART_Transmit(&huart1, (uint8_t *)output, len, 100);
```

OS

- Enable it in ioc



OS

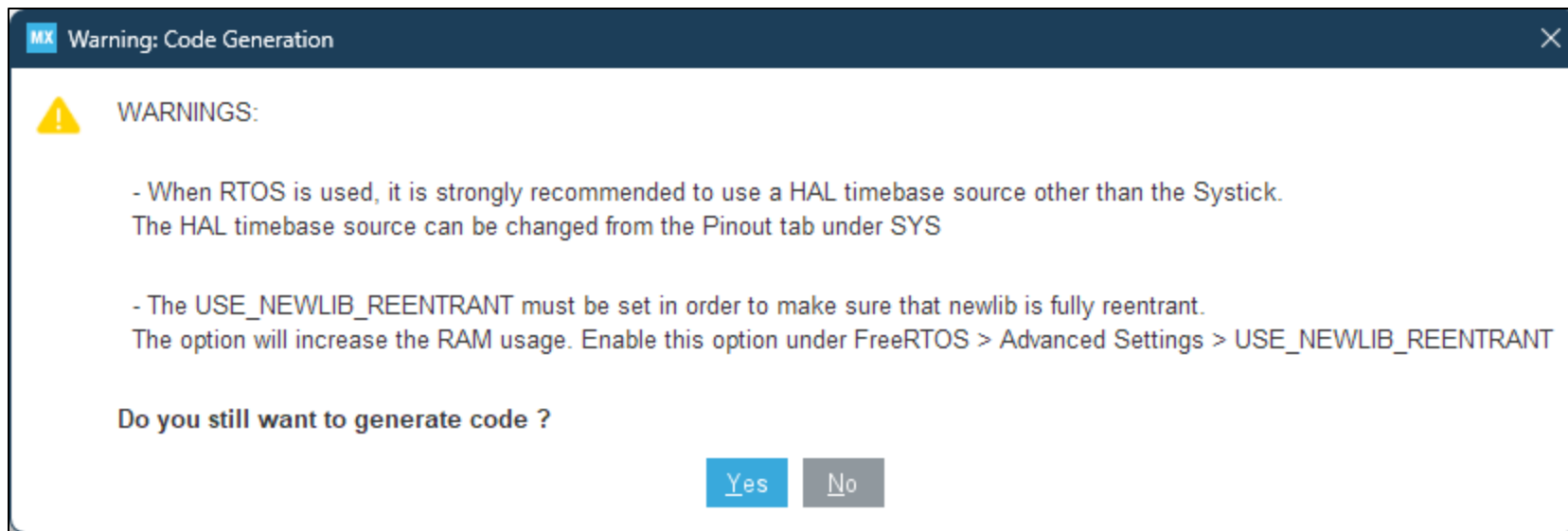
Optional:

checking this box
will create separate
.c and .h files for
each peripheral – it
can make finding
code cleaner if
following good
programming
practices

Pinout & Configuration	Clock Configuration	Project Manager	Tools
Project STM32Cube MCU packages and embedded software packs ☐ Copy all used libraries into the project folder ☒ Copy only the necessary library files ☐ Add necessary library files as reference in the toolchain project configuration file			
Code Generator Generated files ☒ Generate peripheral initialization as a pair of '.c/.h' files per peripheral ☐ Backup previously generated files when re-generating ☒ Keep User Code when re-generating ☒ Delete previously generated files when not re-generated			
Advanced Settings HAL Settings ☐ Set all free pins as analog (to optimize the power consumption) ☐ Enable Full Assert			
User Actions Before Code Generation Browse After Code Generation Browse			
Template Settings Select a template to generate customized code Settings			

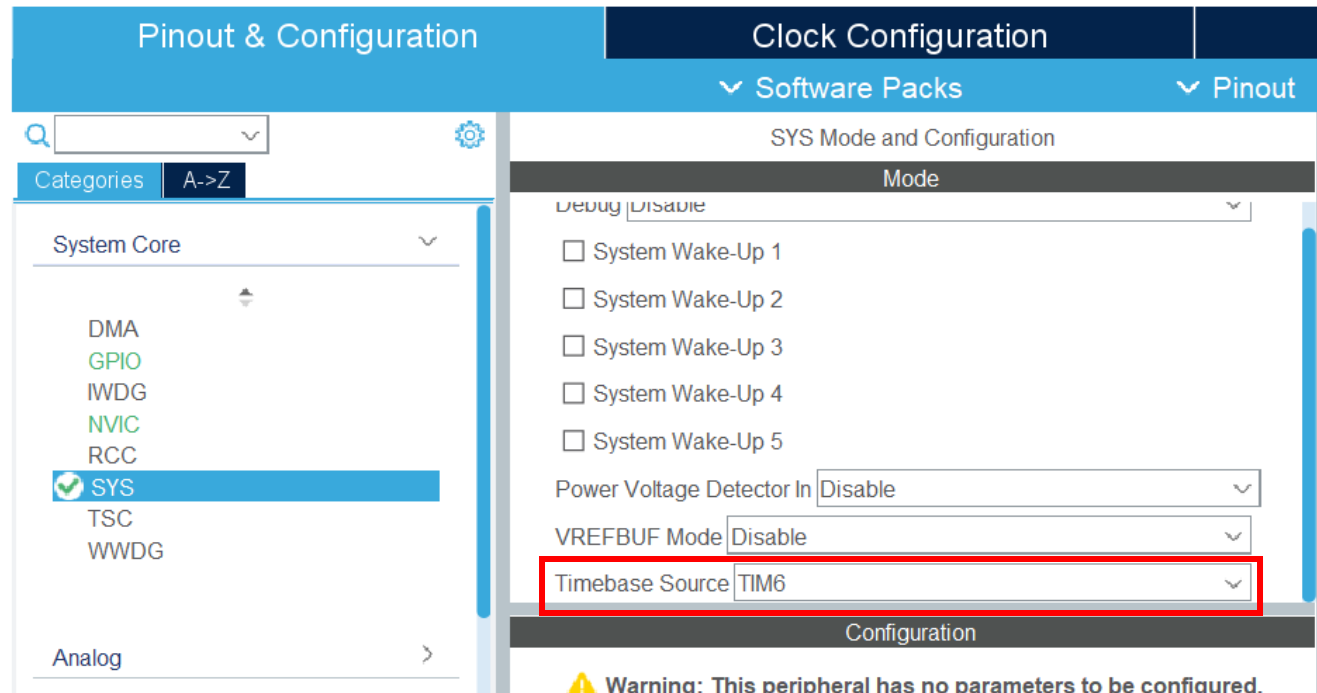
OS

- Uh oh we get a warning!



OS

- Changing the timebase
- osDelay vs HAL_Delay



OS

- Main functionality is no longer in the infinite while

```
131  /* Start scheduler */  
132  osKernelStart();  
133  
134  /* We should never get here as control is now taken by the scheduler */  
135  
136  /* Infinite loop */  
137  /* USER CODE BEGIN WHILE */  
138  while (1)  
139  {  
140      /* USER CODE END WHILE */  
141  
142      /* USER CODE BEGIN 3 */  
143  }  
144  /* USER CODE END 3 */
```

OS

- Put your code in tasks

```
308 /* USER CODE END Header_StartDefaultTask */
309 void StartDefaultTask(void const * argument)
310 {
311     /* USER CODE BEGIN 5 */
312     /* Infinite loop */
313     for(;;)
314     {
315         osDelay(1);
316     }
317     /* USER CODE END 5 */
318 }
```

The screenshot displays the STM32CubeIDE interface with the FREERTOS Mode and Configuration window open. The left sidebar shows the 'Middleware and Software Packs' list, with 'FREERTOS' selected. The main window is divided into 'Mode' and 'Configuration' sections. The 'Configuration' section has a 'Reset Configuration' button and a table of checked options: Events, FreeRTOS Heap Usage, Timers and Semaphores, Mutexes, User Constants, Tasks and Queues, Config parameters, Include parameters, and Advanced settings. Below this is a 'Tasks' table with columns: Task, Priority, Stack, Entry, Code, Para, Alloc, Buffer, and Contr. The table contains one row: 'default' with priority 'osPri...', stack '128', entry 'Start...', code 'Default', para 'NULL', alloc 'Dyna...', buffer 'NULL', and contr 'NULL'. A 'New Task' dialog box is open in the foreground, with fields for Task Name (myTask02), Priority (osPriorityIdle), Stack Size (Words) (128), Entry Function (StartTask02), Code Generation Option (Default), Parameter (NULL), Allocation (Dynamic), Buffer Name (NULL), and Control Block Name (NULL). The 'Add' and 'OK' buttons are highlighted with red boxes.

Middleware and Software Packs

- AIROC-Wi-Fi-Bluetooth-STM32
- FATFS
- FP-ATR-SIGFOX1
- FP-SNS-MOTENVWB1
- FP-SNS-SMARTAG2
- FP-SNS-STBOX1
- FREERTOS**
- I-CUBE-Cesium
- I-CUBE-FS-RTOS
- I-CUBE-ITTIADB
- I-CUBE-Mongoose
- I-CUBE-embOS
- I-CUBE-wolfMQTT
- I-CUBE-wolfSSH
- I-CUBE-wolfSSL
- I-CUBE-wolfTPM
- I-Cube-SoM-uGOAL
- TOUCHSENSING
- USB_DEVICE
- USB_HOST
- X-CUBE-AI
- X-CUBE-ALGOBUILD
- X-CUBE-ALS
- X-CUBE-AZRTOS-L4
- X-CUBE-BLE1
- X-CUBE-BLE2
- X-CUBE-BLEMGR
- X-CUBE-DPower
- X-CUBE-EEPROMA1
- X-CUBE-GNSS1
- X-CUBE-IPS
- X-CUBE-ISPU
- X-CUBE-MEMS1

FREERTOS Mode and Configuration

Mode

Interface: CMSIS_V1

Configuration

Reset Configuration

☒ Events ☒ FreeRTOS Heap Usage

☒ Timers and Semaphores ☒ Mutexes

☒ User Constants ☒ Tasks and Queues

☒ Config parameters ☒ Include parameters ☒ Advanced settings

Tasks

Task	Priority	Stack	Entry	Code	Para	Alloc	Buffer	Contr
default	osPri...	128	Start...	Default	NULL	Dyna...	NULL	NULL

New Task

Task Name: myTask02

Priority: osPriorityIdle

Stack Size (Words): 128

Entry Function: StartTask02

Code Generation Option: Default

Parameter: NULL

Allocation: Dynamic

Buffer Name: NULL

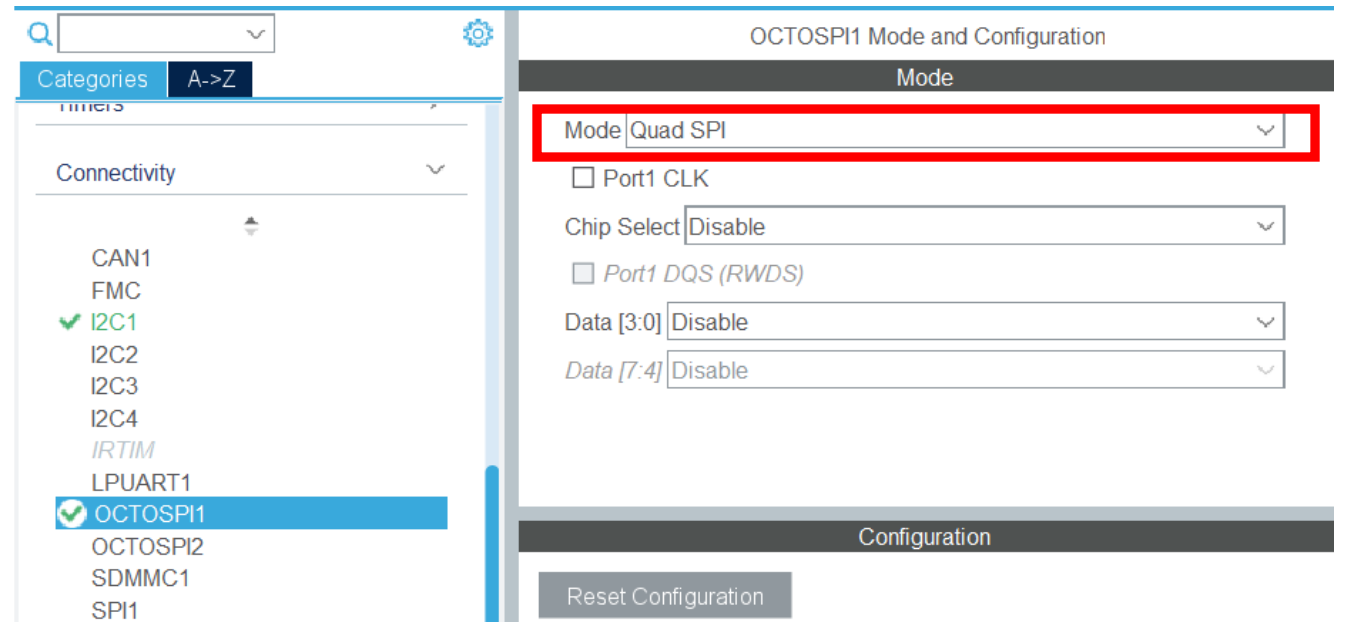
Control Block Name: NULL

Add **Delete**

OK **Cancel**

QSPI

- Set the OCTOSPI1 to Quad SPI mode
- This will not set all (or maybe even any) pins



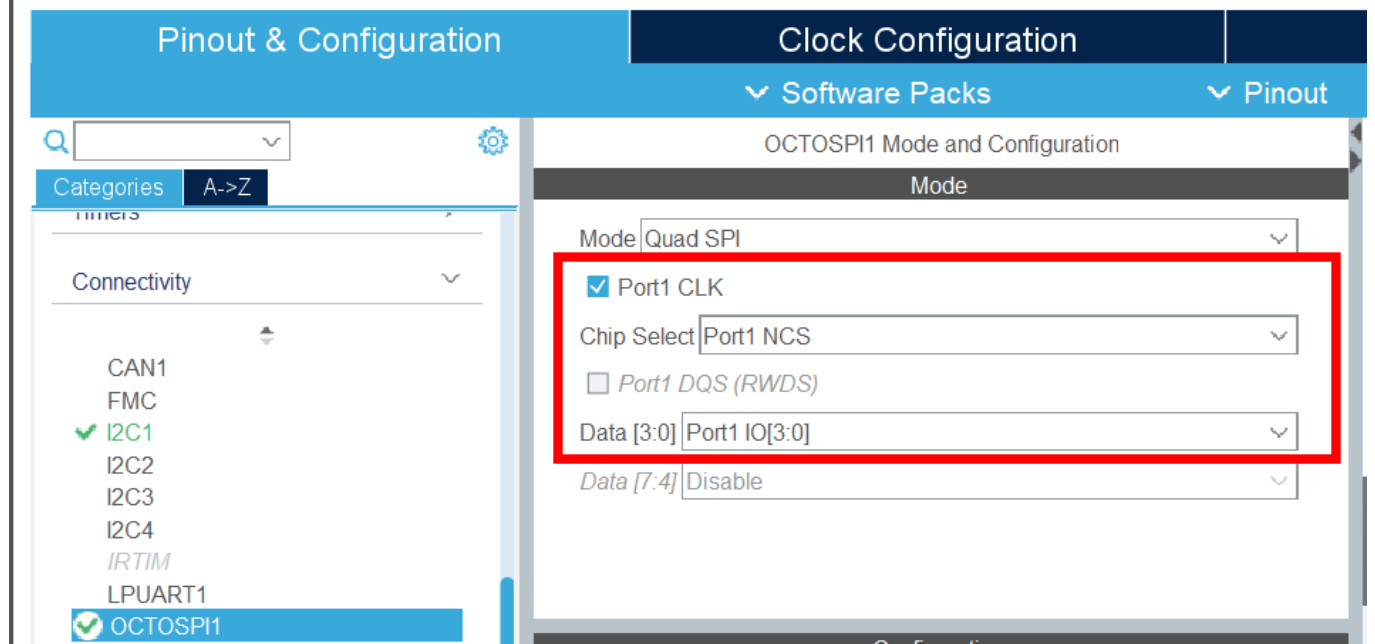
QSPI

- Pinout can be found in this document (for those with the B-L4S5I-IOT01A board): [Discovery kit for IoT node, multi-channel communication with STM32L4+ Series - User manual](#)

Pin number	Pin name	Feature / comment	Signal / label
38	PE7	MEMS microphone	DFSDM1_DATIN2
39	PE8	GPIO_Output	ISM43362-RST
40	PE9	MEMS microphone	DFSDM1_CKOUT
41	PE10	QSPI NOR Flash memory	QUADSPI_CLK

QSPI

- Alternatively this selection does the same thing
- Always double check with the datasheet though!



QSPI

- Once again we have BSP files for handling the QSPI operations

1. Copy .h and .c files from

C:\Users\John\STM32Cube\Repository\STM32Cube_FW_L4_V
1.18.1\Drivers\BSP\B-L4S5I-I0T01

to the project's Core\Inc and Core\Src folders

QSPI

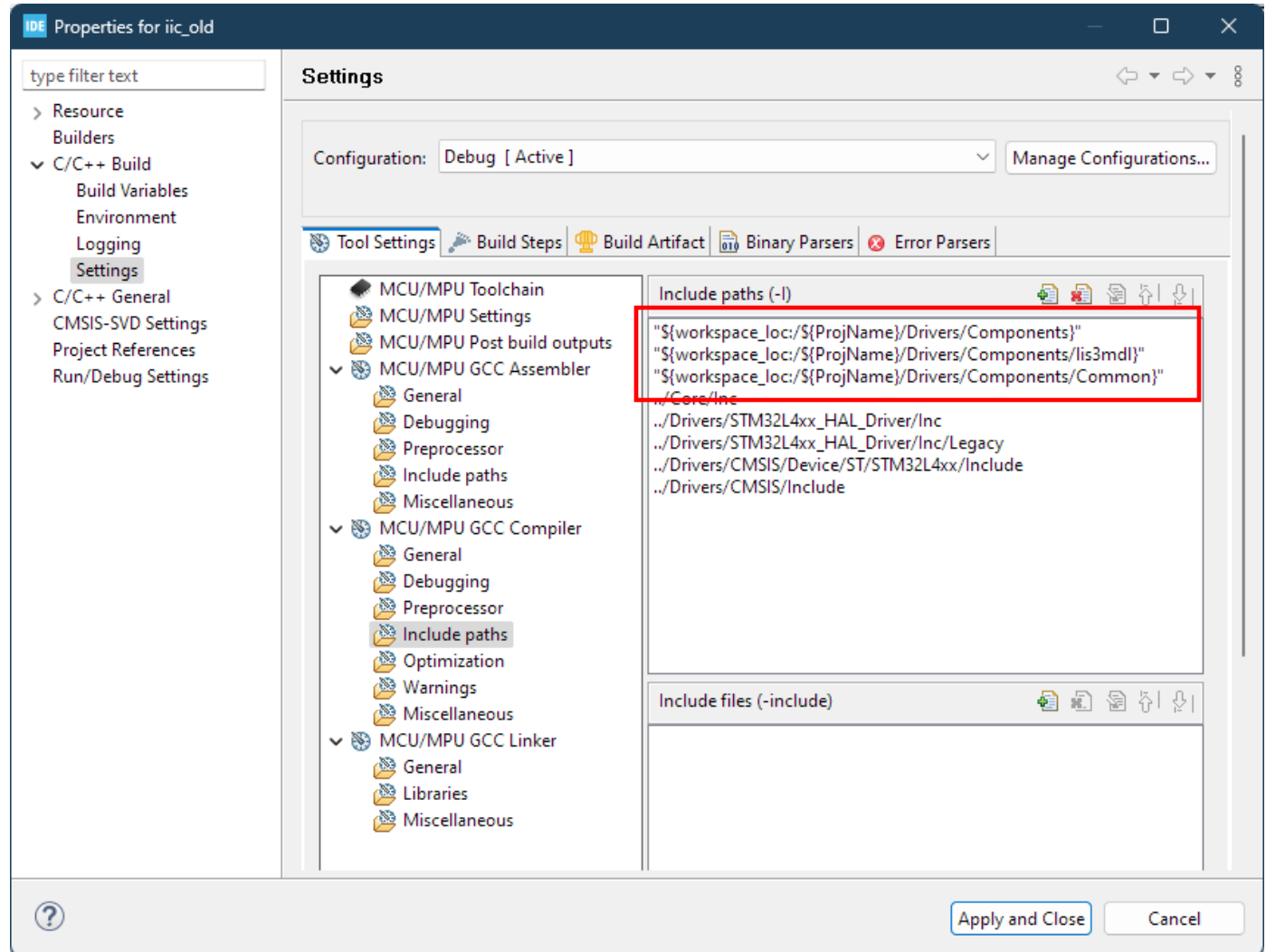
2. Copy mx25r6435f.h from

C:\Users\John\STM32Cube\Repository\STM32Cube_FW_L4_V
1.18.1\Drivers\BSP\Components

to the project's Drivers folders

QSPI

3. Add the files to the linker tree as we did with the sensors in part 1



QSPI

4. Some useful functions you may want to look into

- `BSP_QSPI_Init()` *don't forget to use this one*
- `BSP_QSPI_Write()`
- `BSP_QSPI_Read()`
- `BSP_QSPI_Erase_Block()`
- `BSP_QSPI_Erase_Sector()`

*** `BSP_QSPI_Erase_Sector()` is non-blocking! This means your code will not wait until the function is done executing before continuing ***